CS 300 – Chapter 1 Lectures

C++ Refresher: Arrays, Vectors, and Strings

Jose de Lima

April 23, 2026

Key idea Chapter map

Where this sits in CS 300. Chapter 1 is the *runway*. Every later chapter (linked lists, stacks/queues, trees, hash tables, heaps, graphs) assumes you already reach for `std::vector` without thinking, know when indexing is O(1) and when it isn't, and can trace a loop by hand. If any of that is shaky, the rest of the course compounds the confusion.

What you add to your toolkit here:

- A sharp mental model of **contiguous storage** – arrays and `std::vector` – including what's fast (back-end ops, random access) and what's slow (insert/erase in the middle).
- A library of **iteration patterns**: running best, counter, in-place transform, paired scan – these come back in every later chapter disguised as "traverse the list/tree/graph."
- **Value semantics** in C++: copying, comparing, passing by reference, and the cost of accidentally making a deep copy.
- **C-string survival** so you can read legacy code without panicking, plus the `std::string` / `cctype` upgrades.

Mastery checklist – you should be able to, without references:

1. Write a `for` loop that scans a `vector<int>` keeping a running max *and* the index where it occurred, in one pass.
2. State the Big-O of `push_back`, `pop_back`, `insert(begin(), x)`, `erase(it)`, and `at(i)`, and say *why*.
3. Explain why `resize(n)` and a loop of `push_back` produce the same end state but differ in initial values and in performance.
4. Swap two elements of a vector correctly with and without `std::swap`, and point out why `a = b; b = a;` fails.
5. Convert a small parallel-vectors design into a `vector<Struct>` and explain the tradeoff.
6. Reverse a `vector` in place correctly on the first try (loop bound $n/2$, swap by temp).
7. Read a C-string character-by-character, classify with `cctype`, and explain why `strcmp` returns an `int`, not a `bool`.

Looking ahead. Chapter 2 formalizes *problem-solving* strategies (recursion, greedy, DP); chapter 3 formalizes the cost analysis (Big-O) that you're using intuitively here; chapter 4 introduces the linked-list alternative to vectors, and every performance comparison in that chapter is a comparison *against* what you learned here.

1 1.1 Arrays and Vectors (general concept)

1.1 The motivation: one name, many values

A regular variable holds one value. That is fine when you have a handful of distinct things to track (a wage and a salary), but it falls apart the moment you need a *list*: the four quarter scores of a basketball game, 365 daily temperature readings, every bid in an auction. Nobody wants to declare `score1`, `score2`, `score3`, `score4` – and declaring `temperature_001` through `temperature_365` is absurd.

Definition — Array

A variable that holds a **fixed-length, ordered sequence** of values, all of the same type, under a single name. Each value is called an **element**, and each element has a position in the sequence called its **index**.

A useful analogy: a normal variable is a truck with one cargo bay; an array is a train whose cars are all coupled together and numbered, but which you still call by one name.

1.2 Indexing

The essential feature of an array is **direct access**: given an index, you can read or write the element at that position in constant time, without scanning from the front. You write this with bracket notation in most languages:

```
myArray[2]       // element at index 2
```

Key idea Why direct access is $O(1)$

Array elements are stored *contiguously* in memory. Given the array's starting address and an element's size, the address of element i is simply

$$\text{start} + i \times \text{element_size}.$$

No matter how long the array is, that calculation takes the same amount of time. This is exactly why arrays feel fast, and why random access is the benchmark other data structures are measured against.

Aside The model has a name: Word-RAM

The picture used above – memory is an addressable sequence of fixed-size *words*; the processor reads or writes any one word in $O(1)$ – has a formal name in algorithm analysis: the **Word-RAM** model. Chapter 3 adopts this language when it makes Big-O analysis precise. For now you only need the conclusion: indexed access into contiguous memory is one operation, no matter how big the array.

1.3 Zero-based indexing

In C, C++, Java, Python, and most languages you will meet, the first element is at index **0**, not 1. An array with 4 elements has valid indices `0, 1, 2, 3` – *not* `1, 2, 3, 4`.

This is not a pointless quirk. The index is the *offset* from the start of the array. Element 0 is 0 slots from the start; element 3 is 3 slots from the start. The math in the box above works out cleanly because of it.

Gotcha Off-by-one errors

The most common bug in any language with arrays: treating the length as the last valid index. If `scoresList` has 10 elements, valid indices are `0–9`. Reading `scoresList[10]` is a bug. In C++ this is **undefined behaviour** – no exception, no error at compile time, the program just silently reads whatever memory sits past the array. Sometimes it crashes; sometimes it "works" and corrupts a neighbouring variable. Get in the habit of writing `i < length`, never `i <= length`.

1.4 Arrays vs. vectors

Some languages distinguish between **arrays** (fixed size, set at creation) and **vectors** (resizable, can grow or shrink). Coral uses the word *array* loosely; C++ gives you both:

Example C++ sequence containers

Type	Size	Access
C-style array `int a[10]`	fixed at compile time	`a[i]`
`std::array<int,10>`	fixed at compile time	`a[i]` or `a.at(i)`
`std::vector<int>`	grows at runtime	`v[i]` or `v.at(i)`

`operator[]` does *not* bounds-check; `.at(i)` throws `std::out_of_range` if `i` is invalid. The CS 300 projects use `std::vector` almost exclusively because bid lists grow dynamically as the CSV is read.

Aside Vectors are not linked lists

The word "vector" in C++ does *not* mean a linked list. A `std::vector` is an array that knows how to grow – internally it allocates a contiguous buffer and reallocates a larger one when it runs out of space. Random access stays $O(1)$. We will meet linked lists in Chapter 4 and they behave completely differently.

Key idea The big picture for 1.1

Arrays give you **constant-time access to any element by position**, at the cost of fixed (or managed) size. That single property – direct indexing over contiguous memory – is the reason arrays underpin almost every other data structure in this course. Hash tables use arrays as their bucket storage; heaps use arrays to lay out trees; even `std::vector` is "just" an array with growth built in.

2 1.2 Vectors (in C++)

This is where the course drops the Coral training wheels and starts writing real C++. A `std::vector` is the C++ workhorse container – the one you reach for by default when you need an ordered list – and essentially every CS 300 project uses it.

2.1 Declaring a vector

Two things have to happen before you can use a vector in a source file:

```
#include <vector>     // bring in the vector type
std::vector<int> gameScores(4);   // declare a vector of 4 ints
```

The piece in angle brackets is the **element type** – the kind of value every element will hold. The number in parentheses is the **initial size**.

Definition — Template parameter

`vector` is a *class template* – a blueprint that becomes a concrete type only when you fill in the angle brackets. `vector<int>` is one type, `vector<std::string>` is another, and `vector<Bid>` is another. You will see the same syntax on `std::map`, `std::set`, and every other standard container.

Aside `using namespace std;`

The textbook writes `using namespace std;` at the top of every example so you can say `vector` instead of `std::vector`. It is fine for learning, but in any non-trivial file you should write `std::vector` explicitly. Pulling the whole `std` namespace in globally is a well-known footgun – it can silently collide with identifiers in your own code or in other libraries. The projects in this course are small enough that either convention works, but get used to typing `std::` now.

2.2 Accessing elements

You have two ways to read or write an element:

```
v.at(i)   // bounds-checked: throws std::out_of_range if i is invalid
v[i]      // not bounds-checked: undefined behaviour if i is invalid
```

Gotcha `.at()` vs `[]`

`.at()` is the safer form and what the textbook teaches first – if `i` is out of range you get an exception you can actually catch. `[]` trades safety for speed (no check on every access), which matters in tight inner loops but not in 99% of coursework code. Rule of thumb: `.at()` while you are learning and when the index came from the user; `[]` inside loops where you already guaranteed the bound with `i < v.size()`.

Indices start at **0**, same as arrays. A vector with 4 elements has valid indices `0, 1, 2, 3`. The index can be any non-negative integer expression – a literal, a variable, the result of arithmetic. It cannot be a floating-point value (`v.at(1.0)` will not compile).

Example Nth-oldest lookup

```
std::vector<int> oldestPeople = {122, 119, 117, 117, 116};
int n;                          // user's 1-based pick
std::cin >> n;
if (n >= 1 && n <= 5) {
    std::cout << oldestPeople.at(n - 1);  // translate 1-based -> 0-based
}
```

Note the `n - 1`: the user thinks "the 1st oldest person," but internally that is index `0`. This is one of the most common places off-by-one bugs sneak in.

2.3 Size, looping, and size_t

Every vector knows its current length via `v.size()`. The classic indexed loop over every element:

```
for (std::size_t i = 0; i < v.size(); ++i) {
    std::cout << v.at(i);
}
```

Definition — `std::size_t`

The unsigned integer type the standard library uses for sizes and indices. `v.size()` returns `size_t`, so your loop counter should be `size_t` too. Using a signed `int` instead will draw a "comparison between signed and unsigned" warning on strict compilers. The textbook uses `unsigned int`, which works but is not the idiomatic choice.

For read-only traversals, the modern and much cleaner form is the **range-based for loop**:

```
for (int value : v) {
    std::cout << value;
}
```

No index, no bounds to worry about. Use this whenever you do not actually need `i`.

2.4 Initialisation

Four common patterns:

```
std::vector<int> a(5);                // 5 elements, each 0
std::vector<int> b(5, -1);            // 5 elements, each -1
std::vector<int> c = {3, 1, 4};       // 3 elements, listed explicitly
std::vector<int> d;                   // empty; push_back later
```

Pattern `d` is by far the most common in the projects – you start empty and `push_back` each bid as you read the CSV.

Key idea Vectors grow

Unlike a C-style array, a vector is *resizable*. You can call `v.push_back(x)` to append, `v.pop_back()` to remove the last element, or `v.resize(n)` to change its length. The vector reallocates its internal storage as needed; you do not manage memory by hand. This is the feature that makes `vector` the default choice when you do not yet know how big the list will be.

2.5 Common errors

- **Missing `#include <vector>`** – the compiler does not know what `vector` is, and the error message is spectacularly unhelpful ("ISO C++ forbids declaration of vector with no type").
- **Missing `std::` or `using namespace std;`** – same error, same fix.
- **Out-of-range access with `[]`** – no crash, no diagnostic, wrong data. If you cannot reproduce a bug, suspect this first.

- **Signed/unsigned comparison in the loop** – warning, not error, but occasionally causes an infinite loop if `i` goes negative.

Key idea The big picture for 1.2

A `std::vector` is the modern C++ answer to "I need a list." It combines the $O(1)$ indexed access of an array with the flexibility of dynamic growth, and it manages its own memory. Get fluent in declaration, `push_back`, `size()`, and iteration (both indexed and range-based) – every later data-structure chapter uses `vector` either as a direct tool or as a reference point.

3 1.3 Array / Vector Iteration Drill

The textbook presents this section as three interactive drills with no textual content:

1. find the maximum value,
2. count the negatives,
3. bubble the largest value to the end.

Each drill is a specific instance of a general pattern. Writing those patterns out once makes the drills easier – and more importantly, every iterative algorithm later in the course is a remix of them.

3.1 Pattern 1 – Linear scan with a running "best"

Key idea The pattern

Keep a variable that holds the best value seen so far. Visit every element; whenever the current element beats the running best, update the running best. At the end, the variable holds the answer.

Find the maximum..

```
int maxVal = v.at(0);                // seed with the first element
for (std::size_t i = 1; i < v.size(); ++i) {
    if (v.at(i) > maxVal) {
        maxVal = v.at(i);
    }
}
```

Gotcha Never seed a max with 0

A tempting but broken shortcut is `int maxVal = 0;` before the loop. If *every* value in the vector is negative, you will incorrectly report `0` as the max. Seed with the first element (or with the smallest possible value for the type, `std::numeric_limits<int>::min()`) – not with an arbitrary constant. The same trap applies to finding the minimum: do not seed with `0`.

Find the minimum.. Same pattern, flip the comparison: `if (v.at(i) < minVal)`.

Find the *index* of the maximum.. When you need the position (e.g., for sorting), track `i` instead of the value:

```
std::size_t maxIdx = 0;
for (std::size_t i = 1; i < v.size(); ++i) {
    if (v.at(i) > v.at(maxIdx)) {
        maxIdx = i;
    }
}
```

3.2 Pattern 2 – Linear scan with a running counter

Key idea The pattern

Start a counter at zero. Visit every element; whenever it matches the condition of interest, increment the counter. At the end, the counter holds the total.

Count the negatives..

```
int negCount = 0;
for (std::size_t i = 0; i < v.size(); ++i) {
    if (v.at(i) < 0) {
        ++negCount;
    }
}
```

The same shape solves "how many even numbers," "how many bids above $100," "how many strings start with `A`." The condition changes; the structure does not.

Running sum / running product.. A tiny variation: instead of incrementing by 1, accumulate the element itself. `sum += v.at(i)` gives a total; `product *= v.at(i)` gives a product (seed with 1, not 0).

3.3 Pattern 3 – One selection-sort pass

The third drill asks you to move the largest value to the last position of the array. This is exactly one pass of **selection sort**, and it combines Patterns 1 (find the max index) with a **swap**:

```
// After this loop, the largest value sits at v.back().
std::size_t maxIdx = 0;
for (std::size_t i = 1; i < v.size(); ++i) {
    if (v.at(i) > v.at(maxIdx)) {
        maxIdx = i;
    }
}
std::swap(v.at(maxIdx), v.at(v.size() - 1));
```

Aside Why the swap matters

If you simply *copy* `v.at(maxIdx)` into the last slot, you overwrite whatever was there. The value that *was* at the end is gone. A **swap** preserves it by putting it into the slot the max came from. This is the same reason selection sort is an *in-place* sort: every value stays in the array, just in a new position.

3.4 Range-based form

When you do not need the index, the same three patterns read much more cleanly:

```
int maxVal = v.at(0);
for (int x : v) { if (x > maxVal) maxVal = x; }

int negCount = 0;
for (int x : v) { if (x < 0) ++negCount; }
```

Use the indexed form when you need `i` (e.g., for the swap in Pattern 3) and the range-based form otherwise.

Key idea The big picture for 1.3

Almost every array/vector algorithm you will write is a combination of three building blocks: *scan with a running best*, *scan with a running count or sum*, and *scan-plus-swap*. Internalise the shape of each loop – the initialization, the condition, the update – and you will recognize them on sight in every later algorithm (selection sort, linear search, minimum finding, summing a column, etc.).

4 1.4 Iterating Through Vectors

Section 1.3 covered the *patterns* (running best, counter, scan-plus-swap). This section is about the *loop idiom* those patterns are built on, and about the standard-library tools that often replace the loop entirely.

4.1 Anatomy of the canonical loop

The loop shape you will type thousands of times in C++:

```
for (std::size_t i = 0; i < v.size(); ++i) {
    // work with v.at(i) or v[i]
}
```

Three pieces, each doing one job:

- **Init: `std::size_t i = 0`.**
  Declares the loop counter *inside* the `for` header so it does not leak into the surrounding scope. `size_t` is the type `v.size()` returns; matching it silences the "signed/unsigned comparison" warning. Start at `0` because indices are zero-based.

- **Condition: `i < v.size()`.**
  Continue while `i` is a valid index. Valid indices for a vector of length n are $0, 1, \ldots, n-1$. The last valid index is `v.size() - 1`, not `v.size()`, which is why the condition is strict less-than.

- **Update: `++i`.**
  Advance to the next index. Prefer `++i` to `i++`; see the aside below.

Gotcha < vs. <=

Writing `i <= v.size()` runs the loop one too many times and calls `v.at(v.size())`, which throws `std::out_of_range`. The error looks like:

```
terminate called after throwing an instance of 'std::out_of_range'
  what():  vector::_M_range_check
```

Fix: use <, not <=. Every time.

Aside ++i vs. i++

Both advance `i` by one. The difference is what the expression *evaluates to*: `++i` returns the new value (after increment), `i++` returns a copy of the old value (before increment). For a plain `int` the compiler optimises the copy away and there is no cost difference – but for richer types (iterators, `BigInt`, etc.) `i++` may actually construct a temporary. Building the `++i` habit means you never have to think about it. In a `for` header, where the return value is thrown away, `++i` is strictly the better default.

4.2 Empty loops and infinite loops

Two classic mistakes with the `for` header:

- **Missing condition** – `for (i = 0; ; ++i)`. With no condition the loop never exits on its own. It will eventually walk off the end of the vector and crash.
- **Condition that never becomes false** – `for (i = 0; i < 99; ++i)` over a vector of 365 elements visits only the first 99; a condition referring to something that is not the vector's size is nearly always a bug. Use `v.size()` so the loop adapts to whatever the vector actually is.

Gotcha Reverse iteration with size_t

It is tempting to iterate backwards with `for (std::size_t i = v.size() - 1; i >= 0; -i)`. **This is an infinite loop.** `size_t` is unsigned – it cannot go negative. After `i == 0`, `-i` wraps around to a gigantic positive number. Safe patterns:

```
// Loop while i > 0, then handle 0 separately:
for (std::size_t i = v.size(); i-- > 0; ) {
    // i now takes values size-1, size-2, ..., 0
}
```

or use a signed counter when the index arithmetic needs to go negative.

4.3 Common errors

- **Out-of-range access** with `.at()` throws `std::out_of_range` (program aborts unless you catch it). With `[]` it is silent undefined behaviour. Bounds-check the index whenever it came from the user.
- **Seeding `maxVal` or `minVal` with 0.** Covered in 1.3. Seed with `v.at(0)` instead.
- **Mixing signed and unsigned in the comparison.** `int i` vs. `v.size()` produces warnings and can surprise you if `i` ever goes negative. Use `size_t`.

4.4 The standard-library alternative

Most of the loops in 1.3 and this section have a one-liner equivalent in `<algorithm>` and `<numeric>`. Knowing these saves a surprising amount of noise in your code:

```
#include <algorithm>
#include <numeric>

// Sum of all elements
int total = std::accumulate(v.begin(), v.end(), 0);

// Maximum element (returns an iterator; dereference with *)
int maxVal = *std::max_element(v.begin(), v.end());

// Count elements matching a condition
int negCount = std::count_if(v.begin(), v.end(),
                             [](int x) { return x < 0; });
```

Key idea Iterators, a 30-second preview

`v.begin()` and `v.end()` are **iterators** – lightweight objects that behave like pointers into the container. `begin()` points at the first element; `end()` points *one past* the last (the same "one-past-the-end" convention as `i < v.size()`). Almost every standard-library algorithm takes an iterator pair. You do not need to master iterators now, but recognize the `.begin(), .end()` pattern – it reappears on every STL container.

Aside When to write the loop vs. call the algorithm

The standard-library form is clearer *when the operation has a name* (sum, max, count, sort). The hand-written loop is clearer when you need something more specific – e.g., "sum only the bids above $100 that belong to the closed funds." As a rule of thumb: one-condition scans → algorithm; multi-condition or index-aware work → write the loop.

Key idea The big picture for 1.4

The classic indexed loop – `for (size_t i = 0; i < v.size(); ++i)` – is the default shape for vector iteration. Know every piece of it cold, know the off-by-one and signed/unsigned traps, and know that a handful of standard-library calls (`accumulate`, `max_element`, `count_if`) can replace a lot of handwritten scans when you recognize the pattern.

5 1.5 Multiple Vectors

5.1 The parallel-vectors technique

Sometimes the thing you need to store has more than one field per "row." A standard example pairs country names with the average minutes of TV watched per day in each country:

```
std::vector<std::string> ctryNames(5);
std::vector<int>         ctryMins(5);

ctryNames.at(0) = "China";    ctryMins.at(0) = 155;
ctryNames.at(1) = "Sweden";   ctryMins.at(1) = 154;
ctryNames.at(2) = "Russia";   ctryMins.at(2) = 246;
ctryNames.at(3) = "UK";       ctryMins.at(3) = 216;
ctryNames.at(4) = "USA";      ctryMins.at(4) = 274;
```

The convention: **same index, same row**. Index 3 means "UK, 216 minutes," no matter which vector you look in. To search, you scan one vector for the key and read the matching index in the other:

```
for (std::size_t i = 0; i < ctryNames.size(); ++i) {
    if (ctryNames.at(i) == userCountry) {
        std::cout << ctryMins.at(i) << '\n';
        break;              // early exit once we find it
    }
}
```

5.2 Short-circuiting the search

The textbook uses a flag variable to stop the loop early:

```
bool foundCountry = false;
for (i = 0; (i < ctryNames.size()) && (!foundCountry); ++i) {
    if (ctryNames.at(i) == userCountry) {
        foundCountry = true;
        /* ... */
    }
}
```

This works, but it is not the cleanest form. Two better options:

- **`break`** out of the loop directly – no flag, one less variable to reason about.
- **`std::find`** from `<algorithm>` expresses "scan for a value" in one line:

```
auto it = std::find(ctryNames.begin(), ctryNames.end(), userCountry);
if (it != ctryNames.end()) {
    std::size_t i = it - ctryNames.begin();  // index from iterator
    std::cout << ctryMins.at(i) << '\n';
}
```

Definition — Linear search

Walking a sequence one element at a time until you find the target (or run off the end) is a **linear search**. Its worst-case cost is $O(n)$ – proportional to the size of the vector. For small lists this is fine; for hundreds of thousands of bids, it is painfully slow. A big chunk of CS 300 – hash tables, BSTs – exists to beat this $O(n)$ lookup.

5.3 Why parallel vectors are a smell

Parallel vectors are a reasonable starting technique, but they are **fragile**. Two specific ways they go wrong:

- **They drift out of sync.** Insert into one vector and forget the other; now index 3 means "UK, 246 minutes" instead of "UK, 216." The compiler will not warn you.

- **They do not scale past two or three fields.** When a "row" has ten fields you end up with ten vectors you have to keep in lockstep – and every operation (sort, delete, filter) has to touch all of them.

Gotcha The real answer: a vector of structs

In practice you group related fields into a single **struct** (or class) and keep one vector of those. The TV example rewritten the right way:

```
struct Country {
    std::string name;
    int         minutesTV;
};

std::vector<Country> countries = {
    {"China",  155},
    {"Sweden", 154},
    {"Russia", 246},
    {"UK",     216},
    {"USA",    274},
};

for (const Country& c : countries) {
    if (c.name == userCountry) {
        std::cout << c.minutesTV << '\n';
        break;
    }
}
```

One collection, fields that cannot drift apart, a single sort will move names and minutes together. This is the form the CS 300 projects use: a `Bid` struct with `bidId`, `title`, `fund`, `amount`, stored in a `std::vector<Bid>`. Parallel vectors appear in intro material because structs have not yet been introduced; you will almost never write them outside an intro exercise.

5.4 When parallel vectors are actually okay

Two niches:

- **Temporary scratch data** – e.g., during sorting you sometimes need a parallel "order of original indices" vector.
- **Performance-critical numerical code** – storing each field contiguously ("struct of arrays" layout) can make SIMD and cache behaviour better. This is a microoptimisation you will not need in CS 300.

Key idea The big picture for 1.5

The pattern to learn here is the idea of **linking data across indices**: position i in one collection corresponds to position i in another. Parallel vectors are a primitive, error-prone way to do this. The same idea reappears, better expressed, in a vector of structs ("one row per element"), in a hash table ("key → value"), and in a pair of iterators. Recognize it early.

6 1.6 Vector Resize

6.1 Declaring without a size

The size given in the declaration is not mandatory:

```
std::vector<int> carSales;   // empty: size 0, no elements yet
```

At this point any access – `carSales.at(0)`, `carSales[0]`, `carSales.front()` – is invalid, because there is nothing there. You have to grow the vector first.

6.2 `resize(n)`

`resize(n)` sets the vector's size to exactly `n`.

- If `n` is *larger* than the current size, new elements are appended at the end. They are **default-initialised**: `0` for numeric types, empty string for `std::string`, default-constructed for your own structs/classes.
- If `n` is *smaller*, the extra elements at the end are destroyed. Anything stored past index `n-1` is gone – no warning, no recycle bin.
- If `n` equals the current size, `resize` does nothing.

```
std::vector<int> v;
v.resize(3);     // v is now {0, 0, 0}
v.at(0) = 122;
v.at(1) = 11;
v.at(2) = 7;     // v is {122, 11, 7}

v.resize(2);     // v is {122, 11}; the 7 is gone
v.at(2);         // throws std::out_of_range
```

6.3 `resize()` vs. `push_back()` – which one?

Two different grow-the-vector strategies, and it is worth knowing when each fits.

Example Two ways to read n user values

Resize first (the textbook's preferred style):

```
int n;
std::cin >> n;
v.resize(n);
for (std::size_t i = 0; i < v.size(); ++i) {
    std::cin >> v.at(i);
}
```

Push-back as you go:

```
int n;
std::cin >> n;
for (int i = 0; i < n; ++i) {
    int x;
    std::cin >> x;
```

```
    v.push_back(x);
}
```

Both leave `v` with `n` elements. The difference is who knows the count up front. When the count is known in advance, either works. When the count is *not* known – "read bids until end-of-file" – `push_back` is the only sensible choice, because you cannot call `resize` with a number you do not have yet.

Key idea Rule of thumb

Use `resize(n)` when you know exactly how many elements you want and want them default-initialised in one shot. Use `push_back` when the count is discovered one element at a time (the common case for reading files, parsing input, building results on the fly). CS 300 projects use `push_back` for the bid list.

6.4 Size, capacity, and `reserve()` (what intro texts omit)

A vector tracks *two* numbers, not one:

Definition — Size vs. capacity

- **size** (`v.size()`): how many elements the vector currently *holds*. This is what your loops iterate over.
- **capacity** (`v.capacity()`): how many elements its internal buffer *could* hold before a reallocation is needed. Always $\geq$ size.

When you `push_back` and size is about to exceed capacity, the vector allocates a bigger internal buffer (typically $1.5\times$ or $2\times$ the old capacity), copies everything over, and frees the old buffer. That reallocation is $O(n)$ – but because it happens geometrically less often as the vector grows, the *amortised* cost of `push_back` is $O(1)$.

Definition — Amortized cost

A data-structure operation has **amortized cost** $T(n)$ if *any* sequence of k operations costs at most $k \cdot T(n)$ in total. So $\Theta(1)$ amortized doesn't mean every individual call is cheap – it means the expensive ones happen rarely enough that the average over many calls is bounded. `push_back` fits exactly: most calls write one slot in $O(1)$; the occasional reallocation costs $O(n)$, but a doubling policy means it can only happen once per *geometric* doubling, so the total bill stays $O(k)$ for k pushes. Chapter 3 derives this rigorously.

If you know ahead of time roughly how big the vector will get, you can skip the intermediate reallocations with `reserve`:

```
std::vector<Bid> bids;
bids.reserve(10000);          // capacity -> 10000, size still 0
for (/* each CSV row */) {
    bids.push_back(parseRow(row));  // no reallocations
}
```

Gotcha resize vs. reserve – not the same

- `resize(n)` changes the *size* to `n` (creating or destroying elements as needed). Your loops now iterate `n` times.
- `reserve(n)` only changes the *capacity*; size stays where it was. Nothing to iterate over until you `push_back`.

Mixing them up is a common bug – calling `reserve` when you meant `resize` leaves the vector empty; calling `resize` when you meant `reserve` creates n junk default-constructed elements that your `push_backs` will then *append to*, giving you $2n$ total.

6.5 Related operations

- `v.clear()` – empty the vector (size becomes 0; capacity usually untouched).
- `v.pop_back()` – remove the last element; shrinks size by 1.
- `v.empty()` – returns `true` when size is 0. Prefer `v.empty()` to `v.size() == 0` because on some containers (not `vector`, but good habit) `size()` is not $O(1)$.

Gotcha shrink_to_fit() and the no-auto-shrink policy

`std::vector` grows automatically but *never shrinks automatically.* If you push 10 000 items then pop 9 999, the vector's size is 1 but its capacity is still $\geq$ 10 000 – the unused buffer sits there. `v.shrink_to_fit()` *requests* a release, but the standard explicitly allows the implementation to ignore it (non-binding). To force the buffer down, swap with an empty vector:

```
std::vector<int>().swap(v);   // capacity collapses to 0 (or near it)
```

This contrasts with Python's `list` (and some Java `ArrayList` settings), which actively shrink the backing buffer when too empty. The C++ default prioritizes reusing already-allocated memory over reclaiming it – usually what you want, occasionally not.

Key idea The big picture for 1.6

A `vector` has two knobs: **size** (what you can access) and **capacity** (what has been allocated). `resize` changes size; `reserve` changes only capacity; `push_back` changes size one at a time and lets capacity grow geometrically behind the scenes. Get these three straight and you will avoid most of the "why is my vector empty / doubled / reallocating constantly?" bugs.

7 1.7 Vector push_back, back, and pop_back

Section 1.6 already introduced `push_back` as a way to grow a vector one element at a time. The real value of this section is that it completes the "back of the vector" interface – `push_back`, `back`, `pop_back` – which, taken together, turn a plain `vector` into a perfectly good **stack**.

7.1 The three back-end operations

Definition — Back interface

- `v.push_back(x)` – append `x` to the end; size grows by 1.
- `v.back()` – return a reference to the last element; size is unchanged.
- `v.pop_back()` – remove the last element; size shrinks by 1. Returns `void`.
- `v.front()` – mirror of `back()` for the first element; also size-unchanged.

Every one of these is $O(1)$ (amortised, in `push_back`'s case).

Gotcha `pop_back()` returns `void`

A classic beginner mistake:

```
std::cout << v.pop_back();   // ERROR: pop_back returns void
```

The textbook flags this but does not explain it, so here it is: `pop_back` separates "give me the last value" from "remove the last value." If you want both, call `back()` first, then `pop_back()`:

```
int last = v.back();   // read the value
v.pop_back();          // then drop it
std::cout << last;
```

The reason the standard library splits them is *exception safety*: if returning the value could throw (copy-constructor of a heavy object, out-of-memory), a "pop-and-return" would have to somehow unwind a successful removal. Splitting the operations sidesteps the problem – and happens to be faster too, since `back()` returns a reference without copying.

7.2 Vector as a stack

Key idea Stack via vector

push, top, pop. That is the entire interface of a **stack** – a last-in, first-out container. Rename the three back operations accordingly:

- `push_back` ≡ `push`
- `back` ≡ `top`
- `pop_back` ≡ `pop`

That is why a classic grocery-list example prints items in *reverse* order of entry – "Oranges, Apples, Bread, Juice" comes out as "Juice, Bread, Apples, Oranges." The last item pushed is the first item popped.

In fact the C++ standard library's `std::stack` is a thin wrapper around exactly this pattern. You will meet it formally in Chapter 4.

7.3 Why the back is fast, the front is slow

A common question: why is there no `push_front` in the headline interface?

Key idea Asymmetry of ends

push_back / pop_back are $O(1)$ because they touch only the last slot. Inserting or removing at the *front* would require shifting every other element one slot – that is $O(n)$. A `std::vector` actually does provide an `insert(v.begin(), x)` operation, but it is expensive on purpose. If you need fast front *and* back access, use `std::deque` (double-ended queue) or a linked list, which you will meet in Chapter 4.

This asymmetry is the reason `std::vector` makes a great stack but a poor queue. A queue needs $O(1)$ on both ends; a vector does not offer that.

7.4 Worked example: building a list by reading input

Example The CS 300 pattern

Almost every project in this course builds a vector by reading an unknown number of rows from a file. The shape is:

```
std::vector<Bid> bids;
std::string line;
while (std::getline(file, line)) {
    bids.push_back(parseBid(line));
}
```

Because you do not know up front how many rows the CSV has, you cannot `resize` in advance – push_back in a loop is the natural fit. After the loop, `bids.size()` tells you how many rows were read.

7.5 Small gotchas

- **`back()` on an empty vector** is undefined behaviour – the reference it returns refers to nothing. Always test `!v.empty()` first when you are not sure.
- **`pop_back()` on an empty vector** is also undefined behaviour.
- **A reference from `back()` is invalidated by a later `push_back`** if the push triggers a reallocation. Read the value, use it, then push – do not hold a reference across a push.

Key idea The big picture for 1.7

The push_back / back / pop_back trio gives you $O(1)$ access at the end of a vector and, together, form a stack. Operations at the *front* cost $O(n)$, so vectors are LIFO-friendly and FIFO-hostile – a useful distinction that shapes which data structure you reach for in every later chapter.

8 1.8 Modifying, Copying, and Comparing Vectors

Three distinct topics in one source-text section. The unifying idea, and the thing to walk away with, is that a `std::vector` has **value semantics**: assignment copies the data, equality compares the data, and two vectors that were equal a moment ago can diverge without affecting each other.

8.1 Modifying elements in a loop

You read an element with `v.at(i)`; you write one with `v.at(i) = newValue`. There is nothing special about writing during a loop – the same index works for both directions:

```
// Clamp every negative value to 0.
for (std::size_t i = 0; i < v.size(); ++i) {
    if (v.at(i) < 0) {
        v.at(i) = 0;
    }
}
```

The range-based form. is just as happy – but the loop variable has to be a *reference* or you will be modifying a copy:

```
for (int& x : v) {        // note the &
    if (x < 0) x = 0;
}
```

Gotcha Forgetting the & in a range-based for loop

`for (int x : v)` copies each element into `x`. Writing to `x` changes only the copy; `v` is untouched. Add `&` (`int& x`) to modify in place, or use `const int& x` when you want to read without copying.

8.2 Shifting patterns – the direction matters

The two superficially similar left/right shifts in the textbook exercises produce very different results because of the order the writes happen:

Example Left shift – overwrites ahead of the read

```
for (i = 0; i < 3; ++i) {
    v.at(i) = v.at(i + 1);
}
// {-55, -1, 0, 9}  ->  {-1, 0, 9, 9}
```

Each write happens *before* its source is read, so the shift works correctly from left to right. The last element is duplicated because nothing is to its right to pull from.

Example Right shift – clobbers the data it is about to read

```
for (i = 0; i < 3; ++i) {
    v.at(i + 1) = v.at(i);
}
// {-55, -1, 0, 9}  ->  {-55, -55, -55, -55}
```

Here each iteration writes into the slot the *next* iteration needs to read. The original data gets smeared across the vector. To right-shift correctly you have to walk from the end *backwards*.

Key idea The rule

When modifying a vector in place using its own values, the iteration direction determines whether you read "before" or "after" the write. Walk in the direction that lets you read the original value first, then write. If in doubt, copy to a scratch vector and read from the copy.

8.3 Vector copy with =

Assigning one vector to another copies every element:

```
std::vector<int> origPrices = {10, 20, 30, 40};
std::vector<int> salePrices;

salePrices = origPrices;     // deep copy: salePrices is now {10, 20, 30, 40}
salePrices.at(2) = 27;       // does NOT affect origPrices
```

Definition — Value semantics

A type has **value semantics** when copying a variable copies the data, not a shared reference to it. After `b = a`, mutating `b` has no effect on `a`. `std::vector`, `std::string`, and the built-in numeric types all work this way. Contrast with Python or Java, where assigning one list/array variable to another leaves both names pointing at the same object.

Key idea Why this matters for performance

Copying a vector of n elements costs $O(n)$ time and allocates new memory. That is invisible in a 4-element example but noticeable with a million-row CSV. Two habits to build:

- **Pass by reference** into functions: `void print(const std::vector<int>& v)`. Without the `&`, the whole vector is copied on every call.
- **Use `std::move`** when you are transferring ownership – `b = std::move(a)` avoids the copy by stealing `a`'s internal buffer. `a` is left empty-but-valid.

8.4 Vector comparison with ==

== compares two vectors element-by-element: true if they have the same size *and* every element pair is equal.

```
std::vector<int> x = {3, 4};
std::vector<int> y = {3, 4, 0, 7, 8};
std::vector<int> z = {3, 4, 0, 6, 8};

x == y;   // false: different sizes
y == z;   // false: same size, but index 3 differs
y == y;   // true
```

Aside <, >, <=, >= also work

Not covered in the source text: the relational operators do **lexicographic comparison**, the same way strings are ordered in a dictionary. `{1, 2, 9} < {1, 3, 0}` because at the first differing index the left

side is smaller. Useful when you want to sort a vector of vectors.

Gotcha Never == floating-point vectors by accident

Floating-point arithmetic rounds. `0.1 + 0.2 == 0.3` is `false`. If you compare two `std::vector<double>`s that came from any arithmetic at all, == will often surprise you. Compare element-by-element with a tolerance (`abs(a - b) < eps`) instead.

8.5 Related: does the vector's backing memory move?

Worth knowing, since 1.6 introduced capacity:

- `a = b` always *at minimum* resets `a`'s size. If `b` is larger than `a`'s capacity, the internal buffer is reallocated.
- A `push_back` that crosses the capacity threshold reallocates – any references or iterators you were holding into the vector are now stale.
- Capacity *grows* but does not shrink on its own; see the `shrink_to_fit` warnbox in §1.6.

Key idea The big picture for 1.8

`std::vector` is a value type. = copies, == compares, and modifications to one copy never leak into the other. That makes reasoning about code easy – but it also makes unnecessary copies a real performance cost, so pass by `const&` whenever a function only needs to read a vector.

9 1.9 Swapping Two Variables

Swapping looks trivial until you write it. The instinct `x = y; y = x;` overwrites `x` before its original value is ever used, so both variables end up holding `y`'s value and the old `x` is gone. The fix is to save it somewhere first.

Definition — Swap

To **swap** two variables is to assign each the other's value. If `x = 33` and `y = 55`, after a swap `x = 55` and `y = 33`. A correct swap requires a third location to temporarily hold one value while the other is overwritten.

9.1 Why the naïve version fails

Example The broken swap

```
int x = 33, y = 55;
x = y;   // x is now 55 -- the old 33 is GONE
y = x;   // y is now 55 (the new x), not 33
```

After the first line there is no copy of `33` left anywhere in the program. The second line cannot recover what was just overwritten.

9.2 The temp-variable pattern

Example Correct swap with a temporary

```
int x = 33, y = 55;
int temp = x;   // save x
x = y;          // clobber x -- fine, it's saved
y = temp;       // restore the saved value into y
```

Three lines, one extra variable, always correct. The order matters: save first, then do the overwrite, then use the saved copy.

9.3 C++ shortcut: `std::swap`

C++ gives you the pattern for free in `<utility>` (or `<algorithm>` in older standards), and you should almost always use it instead of writing the temp-variable version by hand.

Example `std::swap`

```
#include <utility>

int x = 33, y = 55;
std::swap(x, y);   // x == 55, y == 33
```

It works on any type that supports move (or copy) assignment: `int`, `std::string`, `std::vector`, your own classes. For `vector` and `string`, `std::swap` is O(1) – it swaps the internal pointers, not the elements.

Gotcha Do not name your variable `swap`

If you write `using namespace std;` and then declare `int swap;`, you shadow the function. Another reason to avoid `using namespace std;` at global scope and to pick more specific variable names.

9.4 Swapping list elements

Most real swaps are between two positions in a vector, not between two free-standing variables. Reversing a vector is the canonical example: swap index `0` with `n-1`, index `1` with `n-2`, and so on, stopping at the middle.

Example Reverse a vector in place

```
#include <vector>
#include <utility>

std::vector<int> v = {10, 20, 30, 40, 50};
for (size_t i = 0; i < v.size() / 2; ++i) {
    std::swap(v[i], v[v.size() - 1 - i]);
}
// v is now {50, 40, 30, 20, 10}
```

The loop runs `n/2` times; integer division handles the odd-length case correctly because the middle element swaps with itself and can be skipped.

Gotcha Off-by-one: do not loop to `v.size()`

If the loop bound is `v.size()` instead of `v.size() / 2`, every element gets swapped twice and the vector returns to its original order. Reversing requires stopping at the midpoint.

Aside `std::reverse` exists

The standard library already has `std::reverse(v.begin(), v.end());` in `<algorithm>`. Writing the loop yourself is useful for learning the pattern, but production code should call the algorithm.

Key idea Big picture

Swap is one of those building blocks that shows up inside almost every sorting algorithm (selection sort, bubble sort, quicksort's partition step, heapsort's sift-down). Understanding why the naïve two-line version fails, and why a third storage slot is required, is the same mental model you will reuse when reasoning about sort correctness later in the course.

10 1.10 Debugging Example: Reversing a Vector

The previous section gave the finished, correct reversal loop. This section walks through the three bugs a first attempt typically hits, in the order they surface, because the *debugging process* is the real lesson. Loop-plus-vector code is where most early C++ bugs live, and the tracing technique used here works for everything later in the course.

10.1 The buggy starting point

Imagine writing this without thinking too hard:

Example Attempt 0 – two bugs in three lines

```
for (size_t i = 0; i < v.size(); ++i) {
    v.at(i) = v.at(v.size() - i);   // "swap"
}
```

There are at least three things wrong here. Each one only becomes visible after you fix the previous one, which is why tracing matters more than guessing.

10.2 Bug 1: off-by-one on the mirror index

Gotcha `v.at(v.size() - i)` on the first iteration

When `i == 0`, the expression is `v.at(v.size())`, which is one past the end. `.at()` responds by throwing `std::out_of_range` and the program aborts. The correct mirror index is `v.size() - 1 - i`.

Aside Why `.at()` is your friend here

If the code had used `v[i]` the program would not have thrown; it would have silently read whatever byte sat past the end of the buffer and kept going. That is undefined behavior. `.at()` turns a silent memory bug into a loud, diagnosable exception. Use it while debugging.

10.3 Bug 2: assignment is not a swap

After fixing the index, the loop runs without crashing but produces the wrong output – values appear duplicated and others disappear:

Example Tracing attempt 1

Input `10 20 30 40 50 60 70 80`, loop body `v.at(i) = v.at(v.size() - 1 - i);`

```
i=0: v[0] = v[7]  ->  80 20 30 40 50 60 70 80
i=1: v[1] = v[6]  ->  80 70 30 40 50 60 70 80
i=2: v[2] = v[5]  ->  80 70 60 40 50 60 70 80
i=3: v[3] = v[4]  ->  80 70 60 50 50 60 70 80   // 40 is gone
i=4: v[4] = v[3]  ->  80 70 60 50 50 60 70 80
```

The original value at `v[0]` was overwritten before it was copied anywhere. This is the same bug as the naïve two-line swap from §1.9: assignment copies one way only.

The fix is the temp-variable swap (or `std::swap`):

```
int tmp = v.at(i);
v.at(i) = v.at(v.size() - 1 - i);
v.at(v.size() - 1 - i) = tmp;
```

10.4 Bug 3: the loop runs twice as long as it should

After fixing the swap, the output comes out identical to the input. The swap is correct this time; the loop just runs too many iterations:

Example Tracing attempt 2

```
i=0: swap v[0] and v[7]  ->  80 20 30 40 50 60 70 10
i=1: swap v[1] and v[6]  ->  80 70 30 40 50 60 20 10
i=2: swap v[2] and v[5]  ->  80 70 60 40 50 30 20 10
i=3: swap v[3] and v[4]  ->  80 70 60 50 40 30 20 10  // reversed
i=4: swap v[4] and v[3]  ->  80 70 60 40 50 30 20 10  // un-reversed
i=5: swap v[5] and v[2]  ->  ...                      // etc.
```

Once the midpoint is crossed, every further swap un-does an earlier one. The loop bound must be `i < v.size() / 2`.

Gotcha Integer division quietly handles the odd-length case

If `v.size()` is 5, `v.size() / 2` is 2. The loop swaps `(0, 4)` and `(1, 3)` and stops; the middle element at index 2 swaps with itself and is correctly left alone. No extra branch needed.

10.5 The debugging methodology, not the answer

The final correct version was already given in §1.9. What matters here is the *technique*:

1. **Read the error.** `out_of_range` pointed straight at an index expression. Don't skip the stack trace.
2. **Trace by hand.** Write out the vector contents after each iteration on paper or in a comment. Bugs 2 and 3 are invisible from the source but obvious after three lines of trace output.
3. **Fix one bug at a time.** Each fix exposes the next bug. Changing three things at once means you will

not know which change did what.

4. **Use `.at()` during development.** Turn silent out-of-bounds into exceptions. Switch to `[]` later if you have measured that bounds checking is a bottleneck (it rarely is).

Key idea Big picture

Reversing a vector is not the skill being taught; the skill is the tight loop of *predict, run, observe, diff*. When a loop does the wrong thing, resist the urge to stare at the code. Instrument it – print the state after each iteration, or step through in a debugger – and compare what you see to what you expected. That habit scales from three-line swap loops all the way up to the sorts, searches, and graph algorithms later in the course.

11 1.11 Arrays vs. Vectors

C++ inherits the raw **C array** and also provides `std::vector`. They look similar on the surface – indexed access, contiguous storage – but they have very different safety and feature profiles. For this course the rule is simple: **use `std::vector` unless a concrete reason forces otherwise**.

11.1 Syntax side-by-side

Example Declaration and access

```
// C-style array: fixed size at compile time
int a[5];                 // 5 ints, uninitialized!
a[0] = 42;
int x = a[0];

// std::vector: size known at runtime, grows if needed
#include <vector>
std::vector<int> v(5);    // 5 ints, all zero-initialized
v.at(0) = 42;             // bounds-checked
int y = v.at(0);
int z = v[0];             // also works, NOT bounds-checked
```

11.2 Why arrays are dangerous

An out-of-range write to a C array does not throw, does not warn, does not even necessarily crash. It just scribbles over whatever happens to live at that memory address.

Gotcha The classic memory-corruption bug

```
int weights[3];
int age = 44;

weights[0] = 122;
weights[1] = 119;
weights[2] = 117;
weights[3] = 199;   // out of range -- writes into age!
```

```
std::cout << age;   // prints 199 (on many compilers)
```

The compiler lays `weights` and `age` next to each other in memory. Writing `weights[3]` lands on `age`'s bytes. Nothing stops it. The program appears correct until something later reads `age` and gets a nonsense value – possibly hours of debugging away from the real bug.

With `std::vector` the same mistake using `.at()` throws `std::out_of_range` immediately, pointing at the exact line.

11.3 Feature comparison

Feature	C array	`std::vector`
Size fixed at	compile time*	runtime
Can grow / shrink	no	yes
Knows its own size	no	yes (`.size()`)
Bounds checking available	no	yes (`.at()`)
Copy with =	no (decays to pointer)	yes (deep copy)
Pass to function	as pointer + length	as value or `const&`
Compare with ==	no (pointer compare)	yes (element-wise)

*C99 variable-length arrays exist but are a GCC extension in C++ and should be avoided.

Definition — Array decay

When a C array is passed to a function or assigned to another variable, it silently converts to a bare pointer to its first element. The size information is **lost**. This is why C-style APIs always pass a separate length parameter (`void f(int* a, size_t n)`), and why `sizeof(a) / sizeof(a[0])` only works in the scope where the array was declared.

11.4 When arrays still show up

You will not eliminate arrays from your C++ code – they appear in places you cannot avoid:

- **String literals** are arrays: `"hello"` is a `const char[6]`.
- `argv` in `main(int argc, char* argv[])` is a C-style array of C-style strings.
- **Legacy C APIs** (POSIX, Win32, most game engines) take pointer-plus-length pairs.
- **Embedded / performance** code sometimes needs the stack-only, zero-overhead guarantee of a raw array.

For the **fixed-size** cases where you would be tempted to use a C array, prefer `std::array<T, N>` – it has a compile-time size like a C array but carries `.size()`, `.at()`, iterators, and value semantics like a vector.

Example `std::array` as the safe fixed-size choice

```
#include <array>

std::array<int, 3> weights = {122, 119, 117};
weights.at(3) = 199;   // throws std::out_of_range
int n = weights.size();  // 3, known at compile time
```

Gotcha `arrayList.size()` does not compile

A common reflex after using `vector` is to write `a.size()` on a C array. Arrays are not objects; they have no member functions. The language-level size of a declared array is `sizeof(a) / sizeof(a[0])`, and even that is wrong after decay. If you want `.size()`, you want `std::array` or `std::vector`.

Key idea Big picture

Arrays are what the hardware actually provides: a contiguous block of memory, indexed by offset, with no bookkeeping. Vectors are what you usually want: the same contiguous block, but with a size, growth, bounds checking, and value semantics layered on top. The extra bookkeeping costs essentially nothing, and the safety is worth it in every course project. Default to `std::vector`; reach for `std::array` when the size is fixed; reach for a raw array only when an external API forces it.

12 1.12 Two-Dimensional Arrays

A 2D array is a table: `R` rows × `C` columns, for `R * C` elements total. The language lets you write `a[r][c]` and pretend you are indexing a grid, but underneath, memory is flat. Knowing how the rows are laid out explains cache performance, how to pass 2D data to functions, and why `std::vector<std::vector<T>` is not actually a 2D array.

Definition — Row-major order

C and C++ store 2D arrays in **row-major** order: all of row 0 first, then all of row 1, then row 2, and so on, in one contiguous block of memory. Given `int a[R][C]`, element `a[r][c]` lives at offset `r * C + c` from the start of the array. The *rightmost* index changes fastest as you walk memory.

12.1 Declaring and using

Example Basic 2D array

```
int grid[2][3];              // 2 rows, 3 cols, 6 ints total

grid[0][0] = 55;
grid[1][2] = 99;

// Brace-initialization makes the shape visible:
int numVals[2][3] = {
    {22, 44, 66},   // row 0
    {97, 98, 99}    // row 1
};
```

12.2 Iterating: the loop order matters

The idiomatic nested loop walks the outer (row) index first, the inner (column) index second:

Example Standard nested loop

```
for (int r = 0; r < R; ++r) {
    for (int c = 0; c < C; ++c) {
        std::cout << grid[r][c] << ' ';
    }
    std::cout << '\n';
}
```

Aside Why row-first is faster than column-first

Walking `r` on the outside and `c` on the inside visits memory *sequentially*: `grid[0][0]`, `grid[0][1]`, `grid[0][2]`, `grid[1][0]`, . . . The CPU prefetches the next cache line before you ask for it. Reversing the loops (`c` outside, `r` inside) jumps by `C` elements each access, thrashing the cache. For a small 3×3 it doesn't matter; for a 10 000×10 000 matrix the two orders can differ by a factor of ten.

12.3 Passing a 2D array to a function

This is where raw 2D arrays bite. The compiler needs to know the row length to compute `r * C + c`, so all dimensions *except the first* must be spelled out in the parameter type:

Example Three equivalent signatures

```
void f(int a[2][3], int rows);   // literal dimensions
void f(int a[][3], int rows);    // first dim can be omitted
void f(int (*a)[3], int rows);   // pointer to row (same thing)
```

Gotcha `void f(int** a)` does NOT take a 2D array

A 2D array is *not* an array of pointers. `int a[3][4]` is 12 contiguous ints, not 3 pointers to 4 ints. Passing it where `int**` is expected compiles only after a reinterpret cast and will then read garbage. If you need a function that accepts any row length, use templates or pass a pointer plus both dimensions.

12.4 The C++ alternative: `std::vector` of `vector`

For runtime-sized 2D data the common idiom is a vector of vectors.

Example 2D as a vector-of-vectors

```
#include <vector>

int R = 4, C = 5;
std::vector<std::vector<int>> grid(R, std::vector<int>(C, 0));

grid[1][2] = 99;          // same syntax as a 2D array
grid.size();              // R
grid[0].size();           // C
```

This is flexible – rows can even have different lengths – but the rows are *not* contiguous in memory. Each inner vector is a separate heap allocation, and cache behavior suffers accordingly.

Key idea Flat 1D storage is often better

For true matrix work the trick is to store one vector of `R * C` elements and compute the index yourself:

```
std::vector<int> grid(R * C);
auto at = [&](int r, int c) -> int& { return grid[r * C + c]; };
at(1, 2) = 99;
```

One allocation, contiguous memory, same big-O as a raw 2D array, plus `.size()` and bounds-checked iteration. This is how most numerical C++ code represents 2D data.

12.5 Higher dimensions

`int a[2][3][5]` declares 30 elements; `int a[100][100][5][3]` declares 150 000. Dimensions multiply, and stack-allocated arrays can blow the stack surprisingly fast. If any dimension is large, allocate with `std::vector` (heap) rather than a raw array.

Key idea Big picture

A 2D array is a convenient fiction the compiler maintains over a flat memory block. Understanding row-major order tells you which loop order is fast, why the column dimensions must be in the function signature, and why `vector<vector<T>` and a `vector<T>` with manual indexing have different performance characteristics despite looking similar. Most course work is small enough that any of these options works; the habits you build here pay off when the matrices grow.

13 1.13 Char Arrays / C Strings

Before `std::string` existed, a **string** in C was just an array of `char`s ending with a special sentinel byte. C++ still supports this representation – it's what you get from string literals, command-line arguments, and every legacy C API you will ever link against – so it is worth understanding even though modern C++ code should use `std::string`.

Definition — Null-terminated string ("C string")

A **C string** is a sequence of `char` values stored in a `char` array, terminated by the **null character** `'\0'` (ASCII code 0). The length of the string is not stored anywhere – it is *determined* by scanning for the null byte. A `char[20]` array can therefore hold strings of length 0 through 19: one slot is always reserved for the terminator.

13.1 Declaring a C string

Example Three equivalent declarations

```
char movie[20] = "Star Wars";  // explicit size, rest zero-filled
char name[] = "Hellen";        // size inferred: 7 ('H','e','l','l','e','n','\0')
char city[5] = {'A','l','a','n','\0'};  // character-by-character
```

In every case the compiler (or you) must make room for the trailing `'\0'`. For the literal `"Amy"`, `char name[4]` is the minimum – three letters plus the null.

Gotcha `char name[3] = "Amy";` silently drops the null

Some older compilers accept this and just don't write a terminator; newer ones will warn or error. Every subsequent `cout « name;` or `strlen(name)` then reads past the end of the array. **Always size the array for the literal *plus one*.**

13.2 Why printing "just works"

`std::cout « some_c_string` does not print 20 characters. It prints characters one at a time until it sees `'\0'`. That convention runs through every C string API – `printf`, `strlen`, `strcpy`, file I/O – which is why a missing or overwritten null byte turns into a memory-corruption bug instead of a tidy error.

13.3 Three common bugs

Gotcha 1. Looping to the array size instead of to the null

```
char s[20] = "hello";
for (int i = 0; i < 20; ++i)  cout << s[i];   // WRONG
```

Prints `hello` then 14 bytes of whatever happens to sit after the visible text. Correct loop:

```
for (int i = 0; s[i] != '\0'; ++i)  cout << s[i];
```

Gotcha 2. Looping past the array entirely

```
for (int i = 0; i < 30; ++i)  cout << s[i];   // UB
```

Reads memory that is not part of `s` at all. May print garbage, may crash, may appear to work. This is undefined behavior.

Gotcha 3. Overwriting the null terminator

```
char name[5] = "Alan";   // {'A','l','a','n','\0'}
name[4] = '!';           // null is gone
cout << name;            // prints "Alan!" then keeps going...
```

The next `cout` walks off the end of the array looking for a `'\0'`. It might hit one quickly (harmless-looking output) or run for a long time printing junk (obvious garbage) – the bug is nondeterministic.

13.4 You cannot assign to a C string

Example The assignment that doesn't assign

```
char title[20] = "Star Wars";
title = "Indiana Jones";   // ERROR: array name is not an lvalue
```

A C array's name is not a variable you can reassign. To *copy* characters into an existing buffer, use `strcpy` or `strncpy` from `<cstring>`:

```
#include <cstring>
std::strcpy(title, "Indiana Jones");  // title must be big enough!
```

Gotcha `strcpy` has no idea how big the destination is

Copying a 30-character string into a `char[20]` compiles cleanly, writes 10 bytes past the end of the array, and silently corrupts memory – classic buffer overflow, historically the root of countless security

vulnerabilities. Use `strncpy`, enforce your own bounds check, or (better) use `std::string`.

13.5 C strings vs. `std::string`

	C string (`char[]`)	`std::string`
Header	`<cstring>`	`<string>`
Stores length	no (scan for `\0`)	yes (`.size()`)
Assignable with =	no	yes
Concat with +	no	yes
Resizes automatically	no	yes
Bounds-checked access	no	`.at()`

Aside `std::string` gives you a C string on demand

Every C++ `string` carries a null-terminated char buffer internally. Call `s.c_str()` to get a `const char*` suitable for passing to a legacy C API:

```
std::string path = "data.txt";
std::FILE* f = std::fopen(path.c_str(), "r");
```

This is how you bridge modern C++ code to C libraries.

Key idea Big picture

The null terminator is one of the oldest and leakiest abstractions in the C language – fast and compact, but every single string operation has to trust that the terminator is where the caller said it would be. Every buffer overflow in the classic sense comes from that trust being broken. `std::string` wraps the same raw buffer with an explicit size, safer copy/assign, and automatic resizing, which is why C++ code uses it almost everywhere. You learn C strings because they are still out there – in `argv`, in POSIX APIs, in file paths – not because you should write new code that uses them.

14 1.14 String Library Functions

Because C strings do not know their own length and do not support = or ==, the standard library ships a family of free functions that take raw `char*` pointers and walk them looking for the null terminator. They all live in `<cstring>` (the C++ name) or `<string.h>` (the C name). You will recognize most of them from any C tutorial.

Definition — The naming convention

Functions starting with `str` work on null-terminated C strings. Functions starting with `mem` work on raw byte buffers with an explicit length. Functions with an `n` in the middle (`strncpy`, `strncat`, `strncmp`) take an extra length argument and are the safer variant of their bare cousin.

14.1 Modifying strings

`strcpy(dst, src)`	Copy `src` (through null) into `dst`.
`strncpy(dst, src, n)`	Copy up to `n` chars.
`strcat(dst, src)`	Append `src` to the end of `dst`.
`strncat(dst, src, n)`	Append up to `n` chars, then a null.

Example strcpy / strcat basics

```
#include <cstring>
char org[100] = "United Nations";
char user[20] = "UNICEF";
char target[10];

std::strcpy(target, user);   // target == "UNICEF"
std::strcat(org, user);      // org    == "United NationsUNICEF"
```

Destination comes first, source second – same order as `target = source;` in regular C++. Swapping the arguments is a classic beginner bug and the compiler cannot catch it.

Gotcha strcpy does not check the destination size

```
char target[10];
std::strcpy(target, org);   // org is 14 chars -- writes 4 past the end!
```

No exception, no error – just silent memory corruption. This is the canonical buffer-overflow bug. Use `strncpy(dst, src, sizeof dst)` if you must, and note even `strncpy` does not guarantee a trailing null when the source is exactly `n` characters long.

Gotcha Array-name assignment still does not copy

```
target = user;     // ERROR: array name is not an lvalue
```

The *only* place = copies a C string is in the declaration itself, because the compiler is initializing the array, not performing a real assignment:

```
char user[20] = "UNICEF";    // OK, compile-time initialization
```

14.2 Inspecting strings

`strlen(s)`	Returns number of chars before the null (type `size_t`).
`strcmp(a, b)`	< 0 if `a < b`, 0 if equal, > 0 if `a > b`.
`strncmp(a, b, n)`	Same, limited to first `n` chars.
`strchr(s, ch)`	Pointer to first `ch` in `s`, or `NULL`.
`strstr(hay, needle)`	Pointer to first occurrence of `needle`, or `NULL`.

Example Length, search, compare

```
size_t n = std::strlen("Alan Turing");     // 11

if (std::strchr(name, ' ') != nullptr) { /* has a space */ }

if (std::strcmp(s1, s2) == 0)  { /* equal */ }
if (std::strcmp(s1, s2) < 0)   { /* s1 comes first */ }
```

14.3 The three strcmp traps

These come up constantly and none of them are syntax errors – the compiler will not catch any of them.

Gotcha 1. Using == on C strings

```
if (s1 == s2) { ... }   // compares POINTER addresses, not content
```

Two character arrays will almost never share an address, so this branch is effectively dead. Silent logic bug.

Gotcha 2. Forgetting `== 0`

```
if (std::strcmp(s1, s2)) { ... }   // backwards!
```

`strcmp` returns **0 when equal**. Treating the result as a boolean reads "truthy means equal," which is the opposite of what it actually means. Always write `== 0` explicitly – your future self, reading the code cold, should not have to remember the convention.

Gotcha 3. `strlen` on a buffer that has no null

```
char buf[10];              // no initializer
size_t n = std::strlen(buf);   // walks memory until it hits any 0 byte
```

Undefined behavior. If you declare a C-string buffer, either initialize it (`char buf[10] = "";`) or do not call string-library functions on it until you have written real content plus a terminator.

14.4 A common pattern: transform characters in place

Example Replace spaces with underscores

```
#include <cstring>
char name[15] = "Alan Turing";

for (size_t i = 0; i < std::strlen(name); ++i) {
    if (name[i] == ' ') name[i] = '_';
}
```

This loop **re-scans the whole string every iteration** because `strlen` is O(n). For long strings, hoist it:

```
size_t len = std::strlen(name);
for (size_t i = 0; i < len; ++i) { ... }
```

Or stop at the null byte directly, which is the idiomatic form:

```
for (size_t i = 0; name[i] != '\0'; ++i) { ... }
```

14.5 The modern C++ translation

Every function in this section has a cleaner `std::string` equivalent that does not buffer-overflow and knows its own size.

C	C++ (`std::string`)
`strcpy(dst, src)`	`dst = src;`
`strcat(dst, src)`	`dst += src;`
`strlen(s)`	`s.size()`
`strcmp(a, b) == 0`	`a == b`
`strcmp(a, b) < 0`	`a < b`
`strchr(s, ch)`	`s.find(ch)`
`strstr(h, n)`	`h.find(n)`

Key idea Big picture

The `cstring` library is doing, function by function, what `std::string` does as member operations. Every bug that `strcpy`, `strcmp`, and `strlen` are famous for (overflow, wrong-direction comparison, missing null) disappears when you let the string type track its own size. Learn these functions so you can read legacy code and glue against C APIs – then write new C++ code with `std::string`.

15 1.15 Char Library Functions: cctype

`<cctype>` is the C++ header for classifying and converting individual `char` values – "is this letter a digit?" "convert this to uppercase." It's tiny, and every function takes an `int`, returns an `int`, and is named `issomething` or `tosomething`. Learn the short list and you have 95% of the character-manipulation toolbox.

Definition — Why it's `cctype` not `ctype`

C headers in C++ gain a leading `c` and lose the `.h`: `<ctype.h>` becomes `<cctype>`, `<string.h>` becomes `<cstring>`, `<stdio.h>` becomes `<cstdio>`. The C++ version puts everything in namespace `std` (so prefer `std::isalpha` over bare `isalpha`).

15.1 Classification: "is this character a ...?"

All of these return an `int` – nonzero for true, 0 for false. You will usually use them in an `if` condition directly.

`isalpha(c)`	alphabetic: `a–z` or `A–Z`
`isdigit(c)`	decimal digit: `0–9`
`isalnum(c)`	`isalpha` or `isdigit`
`isspace(c)`	space, tab, newline, CR, vertical tab, form-feed
`isblank(c)`	just space or tab
`islower(c)` / `isupper(c)`	`a–z` / `A–Z`
`isxdigit(c)`	hex digit: `0–9`, `a–f`, `A–F`
`ispunct(c)`	printable, not space, not alphanumeric
`isprint(c)`	printable (includes space)
`iscntrl(c)`	control char – the complement of `isprint`

Example Classifying characters in a string

```
#include <cctype>
char s[] = "Hey9! Go";

std::isalpha(s[0]);  // 'H' -> nonzero (true)
std::isdigit(s[3]);  // '9' -> nonzero (true)
std::isspace(s[5]);  // ' ' -> nonzero (true)
std::ispunct(s[4]);  // '!' -> nonzero (true)
```

15.2 Conversion: `toupper` and `tolower`

Both take a character and return a (possibly different) character. If the input isn't a letter of the relevant case, the function returns it unchanged – so you can apply it to every character in a string without branching.

Example Uppercase the whole string in place

```
#include <cctype>
char s[] = "Hey9! Go";

for (size_t i = 0; s[i] != '\0'; ++i) {
    s[i] = std::toupper(static_cast<unsigned char>(s[i]));
}
// s is now "HEY9! GO"
```

Gotcha The `unsigned char` cast is not paranoia

`isalpha`, `toupper`, and the rest are specified to take an `int` whose value is either `EOF` (`-1`) or a representation of an `unsigned char`. On platforms where `char` is *signed* (most of them), passing a character with the high bit set – anything above ASCII 127, e.g. accented letters – produces a negative `int` and undefined behavior. Always cast: `std::toupper(static_cast<unsigned char>(c))`. This one line prevents a nasty class of bugs you will only see on non-English input.

15.3 A concrete use case: case-insensitive compare

`strcmp` is case-sensitive. `"Apple"` and `"apple"` are not equal to it. The standard trick is to lowercase both sides and compare.

Example Case-insensitive equality for C strings

```
bool equal_ci(const char* a, const char* b) {
    while (*a && *b) {
        char ca = std::tolower(static_cast<unsigned char>(*a));
        char cb = std::tolower(static_cast<unsigned char>(*b));
        if (ca != cb) return false;
        ++a; ++b;
    }
    return *a == *b;   // both must now be '\0'
}
```

15.4 The C++20 alternative: ranges + lambdas

When you are already using `std::string` (and you should be), the loop gets shorter and the cast disappears into the algorithm.

Example Uppercase with `std::transform`

```
#include <algorithm>
#include <cctype>
#include <string>

std::string s = "Hey9! Go";
std::transform(s.begin(), s.end(), s.begin(),
    [](unsigned char c){ return std::toupper(c); });
// s == "HEY9! GO"
```

The lambda parameter is declared `unsigned char` so the cast happens automatically at the call site – cleaner than scattering `static_cast` through a loop body.

15.5 Parsing pattern: extract digits from a string

This pops up in every intro assignment that asks you to pull a phone number, zip code, or ID out of user input:

Example Keep only digits

```
#include <cctype>
#include <string>

std::string phone = "(603) 555-1212";
std::string digits;
for (unsigned char c : phone) {
    if (std::isdigit(c)) digits.push_back(c);
}
// digits == "6035551212"
```

Key idea Big picture

`cctype` is the character-level counterpart to `cstring`: one handles single chars, the other walks null-terminated arrays of them. Together they cover classic C text processing. Modern C++ adds `std::string`, algorithms, and regular expressions on top, but the `is*` / `to*` functions still show up inside those higher-level tools – so learning the short list here pays off immediately, and does not become obsolete when you graduate to `std::string`.

Summary

Chapter 1 is a whirlwind review of the C++ building blocks every later data-structures topic assumes. The threads worth carrying forward:

- **Prefer safe abstractions.** `std::vector` over raw arrays, `std::string` over `char[]`, `.at()` over `[]` while debugging. Every C-flavored alternative has a silent-failure mode that costs more to debug than the bounds-checked version costs to run.
- **Indices are contracts.** `size_t` is unsigned; `v.size() - 1` underflows catastrophically on an empty vector; a 2D index is `r * C + c` under the hood. Treat every index expression as something to verify, not guess.
- **Copy is not free.** Vector copy is O(n); passing a big container by value does a deep copy. Use `const&` when you only read, `&` when you mutate, and move semantics when ownership transfers. Value semantics are the reason all of this feels sane compared to C.
- **Algorithms over loops.** `accumulate`, `max_element`, `count_if`, `find`, `reverse`, `swap`, `transform` – the `<algorithm>` header already has the loop you were about to write. Reach for it first.
- **Debug by tracing, not by staring.** When a loop produces the wrong output, print the state after every iteration. Three bugs in a ten-line reversal loop were only findable that way.

- **Vectors are the runway for the whole course.** Stacks, queues, hash tables, and heaps will all be built on top of `std::vector` or the indexing pattern it exposes. The habits you lock in here – correct sizing, bounds checking, pass-by-reference, reserve-before-loop – will carry through every remaining chapter.

Aside What this connects to

- **Chapter 2** takes the loops and iteration patterns you learned here and reframes them as *problem-solving strategies* (recursion, greedy, DP). The loops don't change; the way you decide *which loop to write* does.
- **Chapter 3** formalizes the cost analysis you've been doing intuitively ("`push_back` is cheap, `insert` at the front is not"). Big-O is the notation; the underlying mechanisms – shifting, reallocation, amortization – are exactly what you practiced here.
- **Chapter 4** introduces *linked lists* as the alternative to contiguous storage. Every performance argument in that chapter is phrased as "compared to a vector, . . ." – so the more solid your vector intuitions are here, the faster ch. 4 lands.
- **Chapter 5+** (stacks, queues, deques) are built on top of vector and linked-list primitives. The `push_back`/`pop_back` pattern from 1.7 *is* the stack interface.

Companion materials. A one-page cheat sheet for this chapter lives at `chapters/ch_1/notes.tex`; twelve drill prompts you can paste into a fresh Claude session live at `chapters/ch_1/practice.md`. Use the cheat sheet to review, the prompts to test.